

Сравнительный анализ алгоритмов AQM: RED, ARED, Gentle RED

Лабораторная работа

Палагашвили Абули Михайлович

Содержание

1	Цель работы	5
2	Задание	6
2.1	Описание топологии	6
2.2	Конфигурация топологии	7
2.3	Конфигурация AQM	8
2.4	Метрики оценки	8
3	Теоретическое введение	10
3.1	Управление активными очередями (AQM)	10
3.2	RED (Random Early Detection)	10
3.3	ARED (Adaptive RED)	11
3.4	Gentle RED	11
4	Выполнение лабораторной работы	13
4.1	Установка и настройка ns-3	13
4.2	Проведение симуляций	16
4.3	Динамика очереди	16
4.4	Пропускная способность	19
4.5	Сбросы пакетов	21
4.6	Обсуждение результатов	23
5	Выводы	26

Список иллюстраций

4.1	Динамика длины очереди для RED, ARED и Gentle RED	17
4.2	Пропускная способность: сравнение RED, ARED и Gentle RED . . .	20
4.3	Моменты сбросов пакетов для каждого алгоритма	22

Список таблиц

2.1	Параметры топологии и симуляции	7
2.2	Единые пороговые параметры для RED, ARED и Gentle RED	8
4.1	Зависимости ns-3	13
4.2	Статистика длины очереди в установившемся режиме	18
4.3	Доля времени в каждой зоне управления очередью	19
4.4	Статистика пропускной способности (CV — коэффициент вариации)	21
4.5	Статистика сбросов пакетов	23
4.6	Сводное сравнение алгоритмов RED, ARED и Gentle RED	24

1 Цель работы

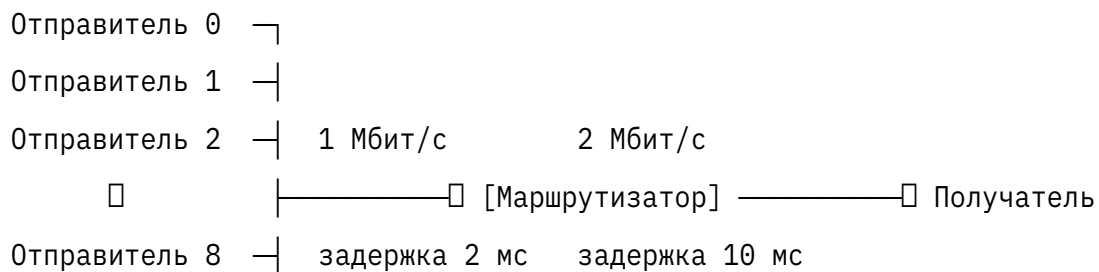
Сравнить три алгоритма активного управления очередями — RED, ARED и Gentle RED — в условиях устойчивой перегрузки узкого места сети и оценить их поведение по трём ключевым метрикам: динамика длины очереди, пропускная способность канала и характер сбросов пакетов.

2 Задание

Провести серию симуляций в ns-3 с топологией «10 параллельных TCP-отправителей → маршрутизатор с AQM → один получатель» при пропускной способности узкого места 2 Мбит/с. Каждый из трёх алгоритмов запустить с одинаковыми параметрами очереди ($\text{minTh} = 5$, $\text{maxTh} = 10$, $\text{MaxSize} = 100$ пакетов) в течение 60 секунд (первая секунда — прогрев, из анализа исключена). Собрать CSV-логи длины очереди, пропускной способности и сбросов пакетов, визуализировать и сравнить результаты.

2.1 Описание топологии

В эксперименте используется топология типа «гантель» (dumbbell): несколько отправителей подключены к общему маршрутизатору через быстрые каналы, а маршрутизатор соединён с единственным получателем через узкий канал с ограниченной пропускной способностью. Узкий канал является единственным местом возможной перегрузки, и именно на его исходящем интерфейсе устанавливается дисциплина очереди AQM.



Отправитель 9 └┐

↑

очередь AQM

(RED / ARED / Gentle)

Каждый из десяти отправителей устанавливает одно TCP-соединение с получателем и непрерывно передаёт данные (BulkSend). Суммарная входящая нагрузка на маршрутизатор потенциально достигает 10×1 Мбит/с = 10 Мбит/с, что в 5 раз превышает пропускную способность узкого канала. Это обеспечивает устойчивую перегрузку и позволяет оценить поведение алгоритмов AQM в реальных условиях насыщения буфера.

Исходящая очередь маршрутизатора (на интерфейсе к получателю) управляется одним из трёх алгоритмов AQM. Параметры топологии приведены в табл. 2.1.

2.2 Конфигурация топологии

Таблица 2.1: Параметры топологии и симуляции

Параметр	Значение	Описание
nSenders	10	Количество TCP-отправителей
fastRate	1Mbps	Скорость каналов отправитель → маршрутизатор (задержка 2 мс)
bottleneckRate	2Mbps	Скорость узкого канала маршрутизатор → получатель
bottleneckDelay	10ms	Задержка узкого канала

Параметр	Значение	Описание
simTime	60.0	Длительность симуляции (с)
Транспорт	TCP BulkSend	Размер сегмента 512 байт

2.3 Конфигурация AQM

Все три алгоритма запускаются с одинаковыми пороговыми параметрами, представленными в табл. 2.2.

Таблица 2.2: Единые пороговые параметры для RED, ARED и Gentle RED

Параметр	Значение	Описание
minTh	5 пакетов	Нижний порог средней очереди — ниже него сбросов нет
maxTh	10 пакетов	Верхний порог; выше него RED/ARED сбрасывают все пакеты
MaxSize	100 пакетов	Жёсткий лимит буфера

2.4 Метрики оценки

Метрика	Источник данных	Что характеризует
Средняя и медианная длина очереди	*_queue.csv	Средняя задержка в буфере
p90 / p99 длины очереди	*_queue.csv	Хвостовые задержки
Доля времени с пустой очередью	*_queue.csv	Недоиспользование канала
Средняя пропускная способность	*_throughput.csv	Утилизация канала
Коэффициент вариации (CV)	*_throughput.csv	Стабильность потока
Общее число сбросов	*_drops.csv	Нагрузка на TCP-механизм перегрузки
Медиана интервала между сбросами	*_drops.csv	Характер управляющих сигналов

3 Теоретическое введение

3.1 Управление активными очередями (AQM)

Традиционный подход к управлению перегрузкой в маршрутизаторах — дроп хвоста (tail-drop): пакеты отбрасываются только тогда, когда буфер полностью заполнен. Это приводит к глобальной синхронизации ТСП-потокa, резким осцилляциям пропускной способности и постоянно переполненному буферу. Алгоритмы активного управления очередями (AQM, Active Queue Management) решают эту проблему, начиная отбрасывать или помечать пакеты *заблаговременно* — до того, как буфер переполнится — на основании статистики средней длины очереди. ТСП-соединения получают сигнал о перегрузке раньше и плавнее снижают скорость отправки, не доводя ситуацию до коллапса.

3.2 RED (Random Early Detection)

RED — базовый алгоритм AQM, предложенный Флойдом и Якобсоном в 1993 году. Алгоритм вычисляет экспоненциально взвешенное скользящее среднее длины очереди (EWMA) и принимает решение о сбросе на его основе.

Обновление средней очереди при каждом поступающем пакете:

$$\bar{q} = (1 - w_q) \bar{q} + w_q q$$

где q — текущая (мгновенная) длина очереди, $w_q \in (0, 1)$ — вес фильтра.

Вероятность сброса определяется тремя зонами:

$$p(\bar{q}) = \begin{cases} 0 & \bar{q} < \min_{Th} \\ \max P \cdot \frac{\bar{q} - \min_{Th}}{\max_{Th} - \min_{Th}} & \min_{Th} \leq \bar{q} < \max_{Th} \\ 1 & \bar{q} \geq \max_{Th} \end{cases}$$

Жёсткий принудительный сброс выше $\max_{Th}$ обеспечивает быстрое реагирование, но синхронизирует TCP-потоки и создаёт пилообразные осцилляции очереди.

3.3 ARED (Adaptive RED)

ARED расширяет RED адаптивным механизмом: параметр $\max P$ периодически пересчитывается через каждые $T_{interval}$ в зависимости от отклонения средней очереди от целевого диапазона:

$$\max P = \begin{cases} \max P + \alpha & \bar{q} > \frac{\min_{Th} + \max_{Th}}{2} \\ \max P - \beta & \bar{q} < \frac{\min_{Th} + \max_{Th}}{2} \end{cases}$$

где α и β — коэффициенты увеличения и снижения вероятности ($\alpha > \beta$). Значение $\max P$ ограничивается допустимым диапазоном. Это позволяет алгоритму приспосабливаться к меняющейся нагрузке без ручной настройки, однако адаптация вносит дополнительную инерцию и может приводить к широким колебаниям очереди.

3.4 Gentle RED

Gentle RED модифицирует поведение RED в зоне выше $\max_{Th}$: вместо немедленного принудительного сброса алгоритм продолжает вероятностный подход

вплоть до $2 \cdot \max_{Th}$:

$$p(\bar{q}) = \begin{cases} 0 & \bar{q} < \min_{Th} \\ \max P \cdot \frac{\bar{q} - \min_{Th}}{\max_{Th} - \min_{Th}} & \min_{Th} \leq \bar{q} < \max_{Th} \\ \max P + (1 - \max P) \cdot \frac{\bar{q} - \max_{Th}}{\max_{Th}} & \max_{Th} \leq \bar{q} < 2 \max_{Th} \\ 1 & \bar{q} \geq 2 \max_{Th} \end{cases}$$

В зоне $[\max_{Th}, 2 \cdot \max_{Th})$ вероятность линейно растёт от $\max P$ до 1, а принудительный сброс происходит лишь при $\bar{q} \geq 2 \cdot \max_{Th}$. Это смягчает реакцию на кратковременные всплески, снижает синхронизацию ТСР-потокaв и уменьшает среднюю длину очереди — ценой несколько большей вариабельности пропускной способности.

4 Выполнение лабораторной работы

4.1 Установка и настройка ns-3

4.1.1 Требования к окружению

Для сборки и запуска ns-3 необходимо наличие следующих компонентов:

Таблица 4.1: Зависимости ns-3

Компонент	Минимальная версия	Назначение
GCC / Clang	9 / 11	Компилятор C++17
CMake	3.13	Система сборки
Python	3.8	Скрипт ./ns3 и вспомогательные утилиты
Git	2.x	Получение исходного кода

На Ubuntu/Debian зависимости устанавливаются командой:

```
sudo apt-get install -y g++ cmake python3 python3-dev \  
git ninja-build gir1.2-glib-2.0 libglib2.0-dev
```

4.1.2 Получение исходного кода

В работе использовался релиз **ns-3.41**. Исходный код загружается с официального репозитория:

```
git clone https://gitlab.com/nsnam/ns-3-dev.git --branch ns-3.41 --depth 1
cd ns-3-dev
```

Альтернативно можно скачать архив релиза с сайта <https://www.nsnam.org/releases/> и распаковать его:

```
tar xjf ns-allinone-3.41.tar.bz2
cd ns-allinone-3.41/ns-3.41
```

4.1.3 Конфигурация и сборка

Конфигурация выполняется через скрипт `./ns3`, который является обёрткой над CMake:

```
./ns3 configure --enable-examples --enable-tests
```

Сборка всех модулей запускается командой:

```
./ns3 build
```

Успешная сборка завершается выводом вида `Build finished successfully`. Для проверки корректности установки запускается встроенный тест:

```
./test.py -s tcp-general
```

4.1.4 Размещение файлов эксперимента

Симуляционные скрипты в ns-3 размещаются в директории `scratch/`. Для данного эксперимента создаётся поддиректория:

```
ns-3.41/
└─ scratch/
```

```
└─ red-experiments/
  │─ red.cc          # главный скрипт симуляции
  │─ configs/
  │   │─ red.txt     # конфигурация RED
  │   │─ ared.txt    # конфигурация ARED
  │   └─ gentle.txt  # конфигурация Gentle RED
  └─ analysis/
      │─ vis.py       # скрипт визуализации
      └─ ...          # CSV-логи и графики
```

Конфигурационные файлы задают тип AQM и параметры очереди. Пример `configs/red.txt`:

```
aqmType=red
minTh=5
maxTh=10
MaxSize=100
```

Аналогичные файлы для ARED (`aqmType=ared`) и Gentle RED (`aqmType=gentle`) отличаются лишь значением поля `aqmType`.

4.1.5 Запуск и проверка сборки скрипта

Перед выполнением серии экспериментов рекомендуется проверить, что скрипт компилируется без ошибок:

```
./ns3 build scratch/red-experiments
```

После успешной компиляции можно переходить к запуску симуляций (см. подраздел ниже).

4.2 Проведение симуляций

Эксперимент состоял из трёх последовательных симуляций — по одной на каждый алгоритм. Для каждого алгоритма создавался отдельный конфигурационный файл (configs/red.txt, configs/ared.txt, configs/gentle.txt) с указанием типа AQM и единых параметров очереди из табл. 2.2.

Симуляции запускались скриптом run_simulations.sh:

```
./ns3 run "scratch/red-experiments/red.cc --config=configs/<алгоритм>.txt"
```

По завершении каждой симуляции три CSV-файла перемещались в директорию analysis/: - <алгоритм>_queue.csv — длина очереди раз в 100 мс; - <алгоритм>_drops.csv — метки времени каждого сброса пакета; - <алгоритм>_throughput.csv — пропускная способность раз в 100 мс.

Для построения графиков использовался скрипт vis.py со сглаживанием скользящим средним (окно 30 отсчётов):

```
python vis.py --algorithms red ared gentle --window 30
```

4.3 Динамика очереди

На рис. 4.1 представлена динамика длины очереди для трёх алгоритмов за всё время симуляции.

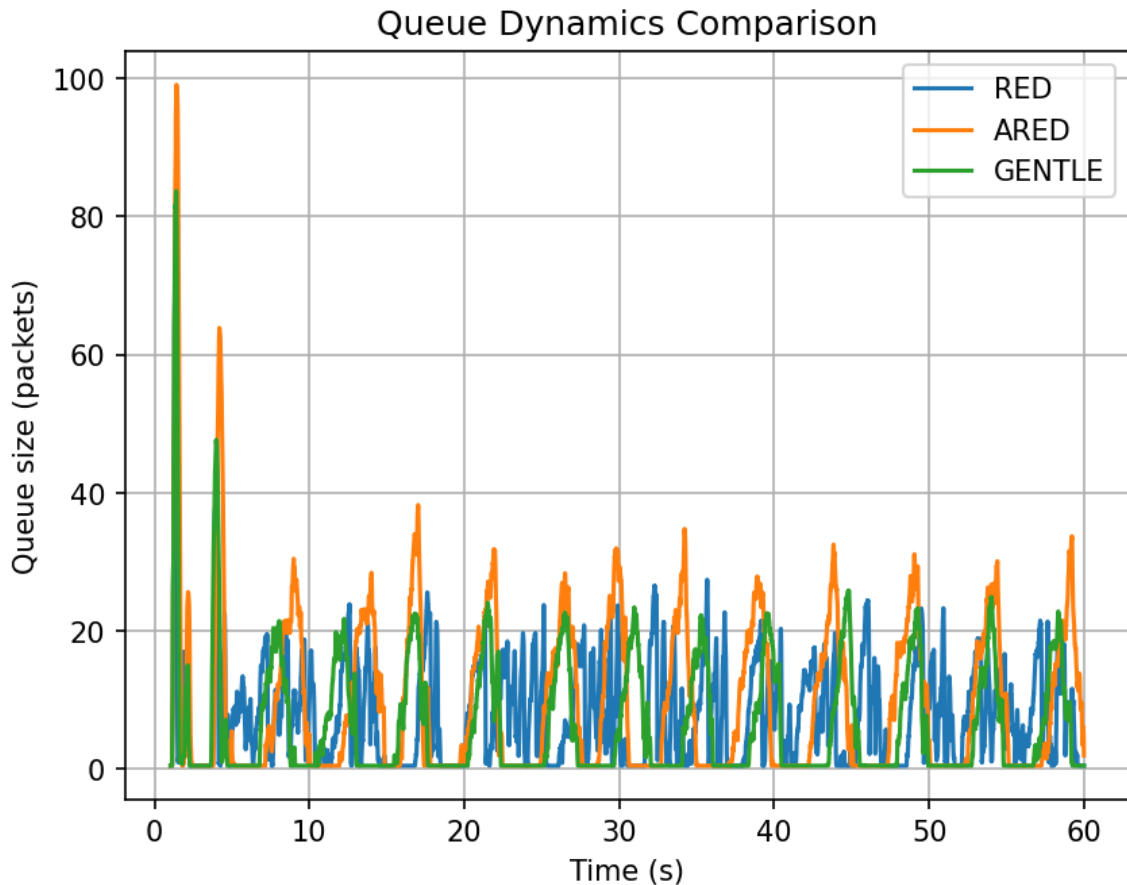


Рисунок 4.1: Динамика длины очереди для RED, ARED и Gentle RED

На графике чётко выделяются две фазы.

Фаза медленного старта ($t = 0-4$ с). Все TCP-соединения одновременно входят в slow-start и за секунды насыщают буфер. Очередь у всех алгоритмов достигает пиковых значений: ARED — до 100 пакетов, RED и Gentle RED — до 84 пакетов. Этот выброс одинаков для всех трёх, поскольку алгоритм AQM ещё не успел накопить статистику и среагировать.

Установившийся режим ($t > 5$ с). После рассасывания начального всплеска алгоритмы расходятся в поведении.

- **RED** (синяя кривая) удерживает очередь в диапазоне 5–26 пакетов с относительно регулярными осцилляциями. Кривая редко касается нуля

(пустая очередь лишь 13.4% времени): принудительный сброс при $m_qAvg \geq \maxTh$ реагирует быстро и держит очередь «в узде», но ценой постоянного давления на ТСП-потoki.

- **ARED** (оранжевая кривая) демонстрирует наибольшую амплитуду колебаний и наиболее хаотичный рисунок. Периодически очередь вырастает до 30–35 пакетов в установившемся режиме и резко обваливается к нулю. Это следствие адаптации $\maxP$: алгоритм поочерёдно «отпускает» и «закручивает» вероятность сброса, порождая широкие и нерегулярные циклы. $p99 = 49$ пакетов — самое высокое значение среди трёх алгоритмов.
- **Gentle RED** (зелёная кривая) чаще остальных возвращается к нулю (26.6% времени очередь пуста, медиана = 1 пакет). Плавная вероятностная функция в зоне $[\maxTh, 2 \cdot \maxTh]$ позволяет ТСП-потокам десинхронизировать свои окна, что приводит к более глубоким, но менее длительным «провалам» очереди.

Статистика установившегося режима приведена в табл. 4.2.

Таблица 4.2: Статистика длины очереди в установившемся режиме

Алгоритм	Сред- нее (пкт)	Стд. откл.	Медиа- на	p90	p99	Пустая (%)
RED	7.74	7.45	6	18	26	13.4%
ARED	9.57	11.94	4	25	49	22.3%
Gentle	6.40	8.86	1	19	34	26.6%

Распределение времени по зонам работы EWMA-средней $\bar{q}$ показано в табл. 4.3.

Таблица 4.3: Доля времени в каждой зоне управления очередью

Алгоритм	$\bar{q} < \min_{Th}$	$[\min_{Th}, \max_{Th})$	$[\max_{Th}, 2 \cdot \max_{Th})$	$\bar{q} \geq 2 \cdot \max_{Th}$
RED	46.0%	19.8%	29.3%	4.8%
ARED	52.6%	8.0%	19.6%	19.7%
Gentle	60.8%	11.8%	19.8%	7.7%

RED проводит 34.1% времени в зоне принудительного сброса ($m_{qAvg} \geq \max_{Th}$), что объясняет его высокую среднюю очередь. ARED проводит 19.7% времени в зоне $m_{qAvg} \geq 2 \cdot \max_{Th}$, где дропаются все пакеты, — отсюда его аномально высокое р99. Gentle RED чаще всего находится ниже $\min_{Th}$ (60.8%) и реже всего попадает в зону жёстких сбросов (7.7%).

4.4 Пропускная способность

График пропускной способности (рис. 4.2) построен с применением скользящего среднего с окном 30 отсчётов, поэтому отображает данные начиная с $t \approx 29$ с. Ось Y намеренно сжата до диапазона 1.74–1.86 Мбит/с, что позволяет рассмотреть различия, невидимые на полной шкале.

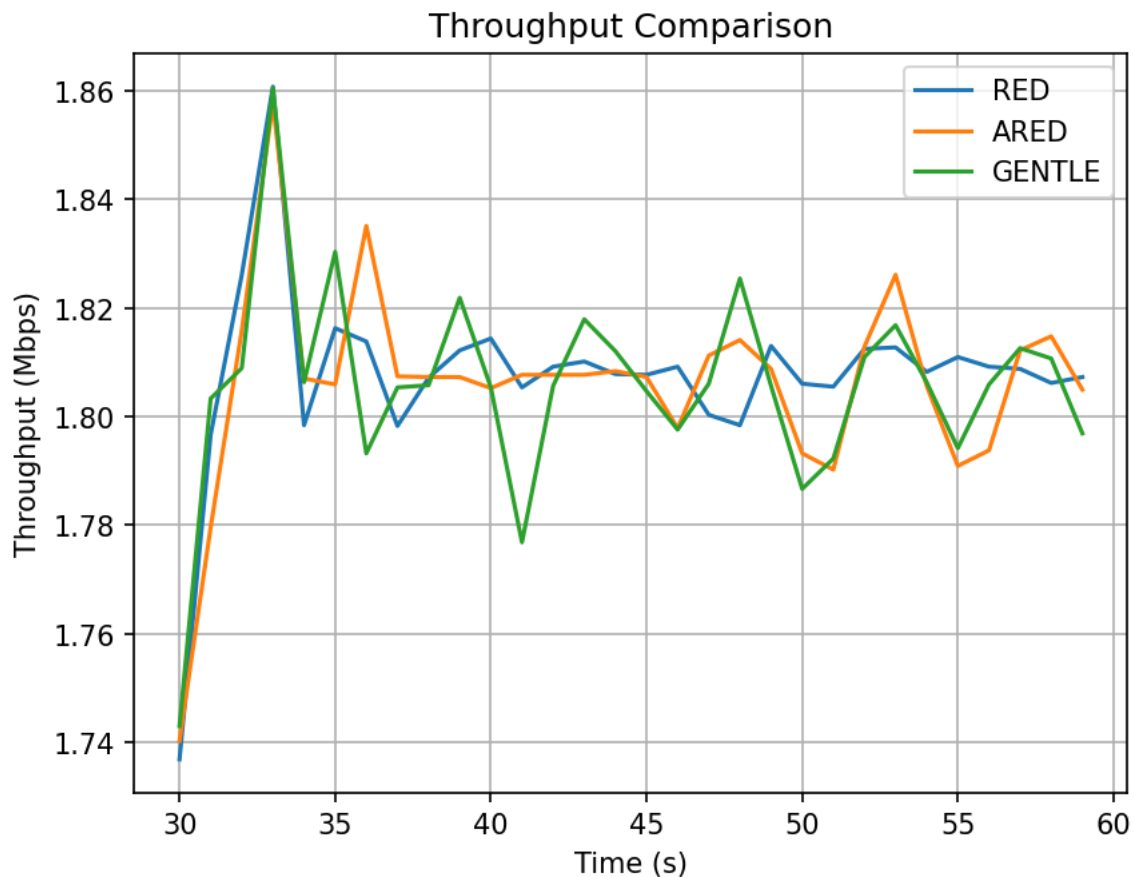


Рисунок 4.2: Пропускная способность: сравнение RED, ARED и Gentle RED

Все три алгоритма стабильно удерживают пропускную способность вблизи **1.80–1.81 Мбит/с** (~90% от номинала канала 2 Мбит/с). Это означает, что выбор алгоритма AQM при данных параметрах не влияет на утилизацию полосы: все три механизма одинаково эффективно «заполняют» канал.

Характерный пик около $t = 33$ с (все кривые достигают ~1.86 Мбит/с) — синхронное восстановление ТСР-потоков после первоначальной волны сбросов: окна конгестии одновременно начинают расти, создавая кратковременный избыточный поток.

Таблица 4.4: Статистика пропускной способности (CV — коэффициент вариации)

Алгоритм	Среднее (Мбит/с)	Стд. откл.	Мин.	Макс.	CV
RED	1.8047	0.3380	0.533	3.621	18.7%
ARED	1.8039	0.3950	0.553	3.293	21.9%
Gentle	1.7998	0.4444	0.352	3.654	24.7%

Несмотря на одинаковое среднее, **RED демонстрирует наименьшую вариативность** (CV = 18.7%): жёсткий синхронный сброс держит все TCP-потoki в такт, делая пропускную способность предсказуемой. Gentle RED имеет наибольший CV (24.7%) — асинхронное управление окнами создаёт большую «рябь», хотя среднее значение от этого не страдает.

4.5 Сбросы пакетов

График (рис. 4.3) представляет моменты сбросов в виде точечной диаграммы: каждая точка — один сброшенный пакет. Три горизонтальные полосы соответствуют трём алгоритмам.

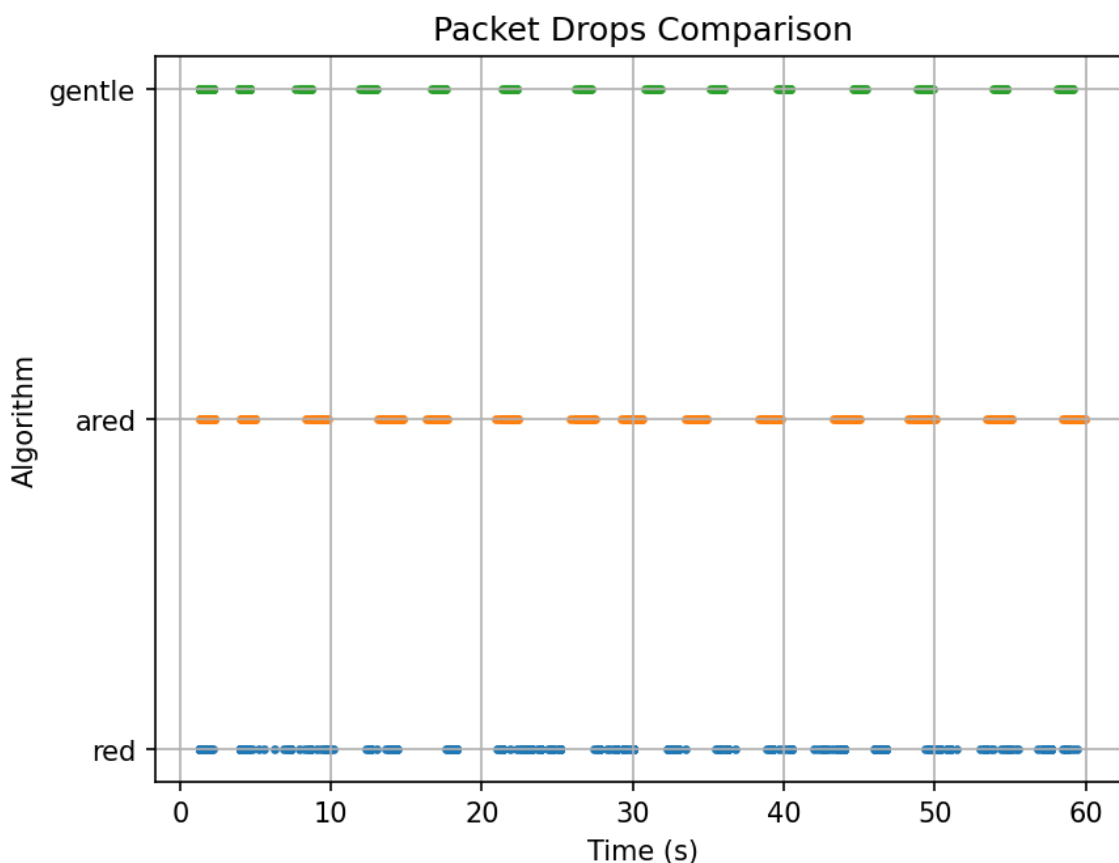


Рисунок 4.3: Моменты сбросов пакетов для каждого алгоритма

RED (нижняя полоса, синий) — самая плотная и непрерывная полоса: точки заполняют практически всё время симуляции, лишь с редкими небольшими просветами. RED активно дропает в 47 из 58 секунд со средним интервалом между сбросами всего **2.3 мс** (медиана). Принудительный сброс при $m_qAvg \geq maxTh$ создаёт непрерывный поток отброшенных пакетов каждый раз, как средняя очередь превышает порог.

ARED (средняя полоса, оранжевый) — чётко видны кластеры сбросов, разделённые заметными паузами. Алгоритм активен в 33 из 58 секунд. Адаптация $maxP$ периодически снижает вероятность сброса до нуля, давая потокам «передышку». Однако когда сбросы происходят, их плотность внутри кластера высока (максимальный всплеск — 276 сбросов в секунду).

Gentle RED (верхняя полоса, зелёный) — наиболее выраженная кластерная структура с самыми длинными паузами между кластерами. Алгоритм активен лишь в 27 из 58 секунд. Внутри каждого кластера сбросы плотные (максимальный всплеск — 305/с), однако между ними — длительные «тихие» промежутки.

Сводная статистика сбросов приведена в табл. 4.5.

Таблица 4.5: Статистика сбросов пакетов

Алгоритм	Всего сбросов	Частота (сбр./с)	Медиана интервала	Активных секунд	Макс. всплеск
RED	1240	21.34	2.3 мс	47	321
ARED	1003	17.11	11.3 мс	33	276
Gentle	1026	17.75	9.1 мс	27	305

RED генерирует на **19–21% больше сбросов**, чем конкуренты. Медиана интервала между сбросами у RED (2.3 мс) в 4–5 раз меньше, чем у ARED и Gentle, — RED практически непрерывно дропает пакеты в периоды перегрузки, тогда как ARED и Gentle работают «импульсами».

4.6 Обсуждение результатов

Итоговое сравнение алгоритмов по всем ключевым метрикам представлено в табл. 4.6.

Таблица 4.6: Сводное сравнение алгоритмов RED, ARED и Gentle RED

Метрика	RED	ARED	Gentle RED
Средняя пропускная способность	1.8047 Мбит/с	1.8039 Мбит/с	1.7998 Мбит/с
CV пропускной способности	18.7% (лучший)	21.9%	24.7%
Средняя очередь	7.74 пкт	9.57 пкт	6.40 пкт (лучший)
Стд. откл. очереди	7.45	11.94	8.86
p99 очереди	26 пкт	49 пкт	34 пкт
Пустая очередь (%)	13.4%	22.3%	26.6%
Всего сбросов	1240	1003 (меньший)	1026
Характер сбросов	непрерывный	кластерный	импульсный
Медиана интервала между сбросами	2.3 мс	11.3 мс	9.1 мс

RED обеспечивает наиболее предсказуемую и стабильную пропускную способность ($CV = 18.7\%$), но достигает этого ценой наибольшего числа сбросов (1240) и наиболее высокой средней очереди. Непрерывный характер сбросов синхронизирует TCP-потоки, создавая жёсткую, «квантованную» динамику управления перегрузкой.

ARED минимизирует число сбросов (1003, -19% к RED) и реже всего тревожит потоки. Однако адаптация $\max P$ создаёт значительную нестабильность очереди (std. = 11.94, $p99 = 49$ пакетов) с периодами чрезмерного роста буфера

— потенциально нежелательными с точки зрения задержки.

Gentle RED достигает наименьшей средней очереди (6.40 пакета) и наибольшей доли пустого буфера (26.6%). Число сбросов сопоставимо с ARED (1026), однако они распределены наиболее компактными импульсами с длинными паузами между ними. Платой является наибольшая вариабельность пропускной способности ($CV = 24.7\%$) вследствие десинхронизации TCP-потоков.

5 Выводы

Проведённый эксперимент показал, что все три алгоритма AQM — RED, ARED и Gentle RED — обеспечивают практически одинаковую среднюю пропускную способность (~1.80 Мбит/с, ~90% от номинала канала), однако принципиально различаются по характеру управления буфером и нагрузке на механизм управления перегрузкой TCP.

Ключевое различие состоит в компромиссе между предсказуемостью и агрессивностью управления очередью. RED действует наиболее агрессивно: принудительный сброс при превышении $\max_{Th}$ удерживает очередь «в узде» (средняя 7.74 пакета) и обеспечивает минимальную вариабельность пропускной способности ($CV = 18.7\%$), но ценой наибольшего числа сбросов (1240) и непрерывного давления на TCP-потoki. ARED, напротив, жертвует стабильностью очереди ради снижения числа сбросов: адаптация $\max_R$ уменьшает сбросы на 19% по сравнению с RED, однако порождает широкие осцилляции (стд. = 11.94, $p_{99} = 49$ пакетов), неприемлемые для приложений, чувствительных к задержке. Gentle RED занимает промежуточное положение, достигая наименьшей средней очереди (6.40 пакета) за счёт более плавной функции сброса, которая десинхронизирует TCP-потoki и снижает заполненность буфера, — но при этом увеличивает вариабельность пропускной способности ($CV = 24.7\%$).

Полученные результаты подтверждают, что выбор алгоритма AQM определяется требованиями конкретного сценария: не существует единственного «лучшего» решения — каждый алгоритм оптимален по своей целевой метрике.

Рекомендации по применению:

- При приоритете **минимальной задержки в буфере** — Gentle RED (наименьшая средняя очередь 6.40 пкт).
- При приоритете **минимального числа сбросов** — ARED (на 19% меньше сбросов, чем RED).
- При приоритете **предсказуемой пропускной способности** — RED (наименьший CV 18.7%).